

IT5002 Computer Systems and Applications
2023/24 Semester I
Final Assessment - ANSWERS

Section A (MCQ) (20 MARKS)

1. Which ONE of the following statements about kernels is true?
 - A. In a monolithic operating system, all the device drivers, memory management, process management, etc. are compiled into one single large program.
 - B. +In a microkernel operating system, only the main kernel is operating at a higher (or highest) privilege level, while device drivers, file systems etc. are operating at a lower privilege level.**
 - C. Microkernel operating systems occupy less memory space than monolithic operating systems.
 - D. Microkernel operating systems run faster than monolithic operating systems.
 - E. None of the above options A. to D. are correct (i.e. Statements A. to D. are ALL FALSE)

2. Which ONE of the following statements about bootstrapping is FALSE?
 - A. The booting process is initiated by a program in read-only memory (ROM) called the Basic Input/Output System (BIOS), or more recently the Unified Extensible Firmware Interface (UEFI).
 - B. By default, the shell is the last major piece of the Operating System to be loaded.
 - C. The bootstrap process results in the file system being loaded.
 - D. +The bootstrap process ends with the system being in a high privilege level.**
 - E. None of the above options A. to D. are correct (i.e. Statements A. to D. are ALL TRUE)

3. Which ONE of the following is NOT a potential source for race conditions between processes or threads?
 - A. +Local variables.**
 - B. Global variables.
 - C. Shared memory.
 - D. A file opened by more than one process.
 - E. None of the above options A. to D. are correct.

4. Which of the following statements listed in i) to v) is/are TRUE about processes?

- i) A program and a process are the same thing.
- ii) A process is responsible for finding its own memory when starting up.
- iii) +The memory allocated to a process can grow or shrink over time.**
- iv) +Processes have their own memory spaces.**
- v) When a process forks a child process, the child process's variables are either re-initialized or contain unknown values.

- A. iii), iv) and v) ONLY are TRUE.
- B. i) and iii) ONLY are TRUE.
- C. +iii) and iv) ONLY are TRUE.**
- D. ii) and iii) ONLY are TRUE.
- E. None of the above options A. to D. are correct.

5. Which of the following statements i) to v) is/are TRUE about process scheduling?

- i) A set of processes will finish faster under Shortest Job First versus First-In-First-Out.
- ii) +The scheduling algorithm used in LINUX ensures as far as possible that a higher priority process will not starve a lower priority process.**
- iii) Given a set of P of processes p_i each with a deadline d_i , a fixed priority scheduling policy ensures that all processes meet their respective deadlines.
- iv) It makes sense to boost processes with large amounts of I/O activity because they are always background processes.
- v) Processes that have used significant amounts of CPU time should be upgraded so that they can finish faster.

- A. ii) and v) ONLY are TRUE.
- B. i) and ii) ONLY are TRUE.
- C. i), iv) and v) ONLY are TRUE.
- D. ii) and iv) ONLY are TRUE.
- E. +None of the above options A. to D. are correct.**

6. During context switching between processes, which of the following i) to v) are saved/restored on the stack?

- i) +General purpose registers.**
- ii) Hardware configuration registers.
- iii) +Program counter.**
- iv) +Status register.**
- v) Open file descriptors.

- A. i) and iv) ONLY
- B. +i), iii) and iv) ONLY**
- C. ii), iv) and v) ONLY

- D. iii) and iv) ONLY
- E. None of the above options A. to D. are correct.

7. Which of the following statements i) to v) is/are TRUE about synchronization mechanisms?

- i) **+Counting semaphores may cause incorrect behavior if used for mutual exclusion.**
- ii) Conditional variables prevent the “lost wakes” problem (i.e. the “producer/consumer problem”) even without mutual exclusion.
- iii) The “lost wakes” problem found in sleep/wake mechanisms can be solved if used with mutexes.
- iv) **+Barriers are useful to wait for all processes to complete a stage before all progress to the next stage.**
- v) **+Semaphores can be used to implement barriers.**

- A. i), ii) iv) ONLY are TRUE.
- B. +i), iv) and v) ONLY are TRUE.**
- C. ii) and iii) ONLY are TRUE.
- D. ii), iii) and v) ONLY are TRUE.
- E. None of the above options A. to D. are correct.

8. Which of the following statements i) to v) is/are TRUE about contiguous memory management, with allocation units of 1 byte?

- i) **+The best-fit strategy may increase the risk of external fragmentation.**
- ii) The worst-fit strategy may increase the risk of internal fragmentation.
- iii) **+It is possible to make the best-fit and worst-fit strategies run in $O(1)$.**
- iv) The buddy allocation algorithm runs in $O(N^2)$.
- v) In the buddy allocation algorithm and two blocks of free memory of size 2^N can be merged to form a new block of size 2^{N+1} .

- A. i), ii) and iii) ONLY are TRUE.
- B. ii) and iv) ONLY are TRUE.
- C. +i) and iii) ONLY are TRUE.**
- D. i) and v) ONLY are TRUE.
- E. None of the above options A. to D. are correct.

9. Assuming a FAT32 filesystem with a logical block size of 4KB. What is the largest partition/disk size that this file system can support? (1 GB = 1024 MB, 1 MB = 1024 KB, 1 KB = 1024 bytes)

- A. 512 GB
- B. +1024 GB**
- C. 8192 GB
- D. 16384 GB
- E. None of the above options A. to D. are correct.

10. Which ONE of the following statements about file systems is TRUE?

- A. The minimum information needed in the directory for a contiguous file system is the starting block and number of blocks in the file.
- B. We can store the metadata for a file in the FAT.
- C. There are sections of the partition that are specially reserved for subdirectories.
- D. +In a linked-list file system it is possible to organize the directory so that access to the first byte and last byte of a file can be accessed in $O(1)$ time regardless of file size.**
- E. None of the above statements A. to D. are true.

11. Suppose that we have two processes A and B with a single global variable x in shared memory. x has an initial value of 2. Each process modifies x as shown below:

Process A: $x = x + 2;$

Process B: $x = x * 2;$

The processes run on a machine with a load-store architecture.

- a. Assuming that there are no race conditions, how many possible values of x are there after all three processes execute their respective statements? (For example, If you find that the possible values of x are 3, 4, 4, 6, – this counts as 4 possible values)

2 values depending on the sequence of execution:

A, B: 8

B, A: 6

- b. Suppose now that there is a race condition. How many possible values of x are there after both processes execute their respective statements? (For example, If you find that the possible values of x are 3, 4, 4, 6, – this counts as 4 possible values)

For a load/store architecture only the load/store interleaving affects the final outcome. We call these LA, SA, LB, SB for load/store in A and B respectively.

Start with LA: Then there are only three interleavings for B:

LA, SA, LB, SB = 8

LA LB SA, SB = 4

LA, LB, SB, SA = 4

Similarly if we start with LB: There are only three interleavings for A:

LB, SB, LA, SA = 6

LB, LA, SB, SA = 4

LB, LA, SA, SB = 4

So this gives 6 possible values.

12. We have three processes P1, P2 and P3 shown below running on a single CPU which must be shared amongst them.

Process	Ready to run time	CPU time	Priority (part b)
P1	1	5	Lowest
P2	0	8	Middle
P3	3	4	Highest

- a. In a system with a round-robin scheduling policy where processes get to run for **3 cycles** each time, how many cycles would each process take, in total, to run? (If a process was ready to run at time 2, and ended running at time 17, it has taken $17 - 2 + 1 = 16$ cycles).

[illegible]

- b. In a system with a fixed-priority preemptive scheduling policy with P1 having the LOWEST priority and P3 the HIGHEST, how many cycles would each process take, in total, to run? (If a process was ready to run at time 2, and ended running at time 17, it has taken $17 - 2 + 1 = 16$ cycles).

[illegible]

- c. In a system with a shortest-remaining-time non-preemptive scheduling policy where each process gets to run for **2 cycles** each time it is picked, how many cycles would each process take, in total, to run? (If a process was ready to run at time 2, and ended running at time 17, it has taken $17 - 2 + 1 = 16$ cycles).

[illegible]

13. We consider a system that allocates memory in units of 4 bytes (i.e. each allocation unit or AU is 4 bytes) and byte addresses. The system uses a bitmap in an array called freeMemory, where each free AU is marked with a “1” bit, and each used AU is marked with a “0” bit. The freeMemory array is 16 bytes long. The first bit is for the AU starting at address 0, the next bit is for the AU starting at address 4, etc.

- a. What is the maximum amount of memory in bytes that this system can manage?

Every bit represents 4 bytes. There are $16 \times 8 = 128 \times 4 =$ bits representing 512 bytes”
 Answer: 512

Suppose now that the freeMemory array contains the following values in hexadecimal:

0x3F 0xF0 0x0E 0x78
 0x00 0x00 0x00 0x00
 0x00 0x00 0x00 0x00
 0x00 0x00 0x00 0x00

This is equal to:

00111111 11110000 00001110 01111000
 00000000 00000000 00000000 00000000
 00000000 00000000 00000000 00000000
 00000000 00000000 00000000 00000000

We have the following memory requests (remember that we are now allocating in units of 4 bytes).

1. a = malloc(14) // Allocate 14 bytes
2. b = malloc(20) // Allocate 20 bytes
3. free(a) // Free request in 1.
4. c = malloc(9) // Allocate 9 bytes
5. d = malloc(8) // Allocate 8 bytes
6. free(b) // Free request in 2
7. e = malloc(18) // Allocate 18 bytes

Suppose we allocate using a worst-fit policy:

Operation	Starting bitmap	Allocated address	Ending bitmap	Internal Fragmentation
a = malloc(14) 4 AU	00111111 11110000 00001110 01111000	8	00000011 11110000 00001110 01111000	2

b = malloc(20) 5 AU	00000011 11110000 00001110 01111000	24	00000000 00010000 00001110 01111000	0
free(a)	00000000 00010000 00001110 01111000	-	00111100 00010000 00001110 01111000	-
c = malloc(9) 3 AU	00111100 00010000 00001110 01111000	8	00000100 00010000 00001110 01111000	3
d = malloc(8) 2 AU	00000100 00010000 00001110 01111000	100	00000100 00010000 00001110 00011000	0
free(b)	00000100 00010000 00001110 00011000	-	00000111 11110000 00001110 00011000	-
e = malloc(18) 5 AU	00000111 11110000 00001110 00011000	20	00000000 00110000 00001110 00011000	2

- b. What are the starting addresses of the memory blocks requested in steps 1, 2, 4, 5 and 7?

Step 1: 8
Step 2: 24
Step 4: 8
Step 5: 100
Step 7: 20

- c. What are the new values in freeMemory after step 7? State your answers in hexadecimal.

0x00 0x30 0x0E 0x18
0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00

- d. What is the total internal fragmentation in bytes after step 7?

Allocation is in units of 4 bytes, so internal fragmentation is just the closest unit of 4 bytes minus the request. Total:

1: 2 bytes
2: 0 bytes
4: 2 bytes
5: 0 bytes

7: 3 bytes

Total = 2 + 0 + 2 + 0 + 3 – 2 – 0 = 5 bytes

14. We now have a buddy allocation system with a total of 1024 bytes of memory. We repeat the same requests from Question 13, shown here again for your convenience. As before we are using allocation units (AUs) of 4 bytes. Thus free memory entries at entry *avail[i]* indicate free memory blocks of 2^i AUs, and not 2^i bytes.

Where there are two suitable blocks of the same size, the block with the lower starting address is always chosen.

1. a = malloc(14) // Allocate 14 bytes
2. b = malloc(20) // Allocate 20 bytes
3. free(a) // Free request in 1.
4. c = malloc(9) // Allocate 9 bytes
5. d = malloc(8) // Allocate 8 bytes
6. free(b) // Free request in 2
7. e = malloc(18) // Allocate 18 bytes

- a. How many AUs are there in this system?

Number of AUs = 1024 / 4 = 256 AUs

- b. What are the starting addresses of the memory blocks in steps 1, 2, 4, 5 and 7?

Request	Starting Address	Actual # of AU/bytes	Internal Frag
a = malloc(14) 4 AU	0	4/16	2
b = malloc(20) 5 AU	8	8/32	12
free(a)	-	-	-
c = malloc(9) 3 AU	0	4/16	7
d = malloc(8) 2 AU	4	2/8	0
free(b)	-	-	-
e = malloc(18) 5 AU	8	8/32	14

Step 1: 0

Step 2: 8

Step 4: 0

Step 5: 4

Step 7: 8

- c. At the end of step 7, what is the total internal fragmentation in bytes?

$$2 + 12 + 7 + 0 + 14 - 2 - 12 = 21 \text{ bytes}$$

15. We have an inode-based file system where the logical block size is 2048 bytes (2 KB). The first 32 blocks of the file system are reserved for various data structures like the boot block, directory, etc. In particular blocks 16 to 31 hold the bitmap that indicates free data blocks with a "1", and occupied data blocks with a "0". Data blocks begin at block 32, so the very first bit of the bitmap indicates the status of block 32, the second bit indicates the status of block 33, etc.

Consider the following example of the **first 8 bits** of this bitmap:

0	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---

As data blocks begin at block 32, this means that blocks 32 to 34 and 39 are occupied, while blocks 35 to 38 are free.

- a. What is the maximum disk or partition size possible on this file system excluding the blocks reserved by the file system? Express your answer in decimal in MB (1 MB = 1024 KB, 1 KB = 1024 bytes)

$$\text{Total number of blocks for bitmap} = 31 - 16$$

$$\text{Total number of bytes} = 16 \times 2048 = 32768 \text{ bytes}$$

$$\text{Total number of blocks} = 32768 \times 8 = 262144$$

Each block is 2KB

$$\text{Total size} = 512 \text{ MB}$$

- b. What are the first and last data block numbers of the data blocks represented by this bitmap?

$$\text{Total number of blocks for bitmap} = 31 - 16$$

$$\text{Total number of bytes} = 16 \times 2048 = 32768 \text{ bytes}$$

$$\text{Total number of blocks} = 32768 \times 8 = 262144$$

$$\text{Starting block number} = 32$$

$$\text{Ending block number} = 32 + 262144 - 1 =$$

$$\text{Block } 262175$$

The first 16 bytes of the free blocks bitmap is shown below in binary:

```
0b00000000 0b00000011 0b11111100 0b00001100
0b00011111 0b11111111 0b00001100 0b00000000
0b11001111 0b00000000 0b11110000 0b00001111
0b11111111 0b10101010 0b00001111 0b10101111
```

Our OS provides the following calls to create/open, read, write, and close a file, as well as to move to a particular byte in the file.

`fd = open(filename)`: Opens a file for reading and writing, returning a file descriptor to `fd`

`lseek(fd, bytenum, from)`: Moves the file pointer to byte number `bytenum`, which can be negative. The “from” argument can take three values:

START: Move to `bytenum` bytes from the start of the file

CURR: Move to `bytenum` bytes from the current file pointer position.

`read(fd, buffer, count)`: Read `count` bytes from file into `buffer`.

`write(fd, buffer, count)`: Write `count` bytes from `buffer` to file.

`close(fd)`: Close the file

The inode and index and data blocks are allocated using the free blocks bitmap on a “first available block” basis. So if we are creating a file, we need to:

- i) Find the first free data block for the inode.
- ii) Create an entry in the directory with the filename and inode block number.
- iii) Find further free blocks for the data, and for lower level index blocks if necessary.

Consider the following program, where `test.txt` does not exist yet. Our free block bitmap is as shown earlier:

```
buffer=[0]
buffer1="Hello"
buffer2= "world."

fd = open("test.txt");

for i in range(1024):
    write(fd, buffer1, 5);

lseek(fd, 2141, CURR)

for i in range(128):
    write(fd, buffer2, 6);
```

- c. The directory entry for the filesystem just consists of the filename and the inode block number. Fill in both fields below:

```
0b00000000 0b00000011 0b11111100 0b00001100
0b00011111 0b11111111 0b00001100 0b00000000
0b11001111 0b00000000 0b11110000 0b00001111
0b11111111 0b10101010 0b00001111 0b10101111
```

First free block is block 46

Filename: test.txt

inode block number: 46

End map:

```
0b00000000 0b00000001 0b11111100 0b00001100
0b00011111 0b11111111 0b00001100 0b00000000
0b11001111 0b00000000 0b11110000 0b00001111
0b11111111 0b10101010 0b00001111 0b10101111
```

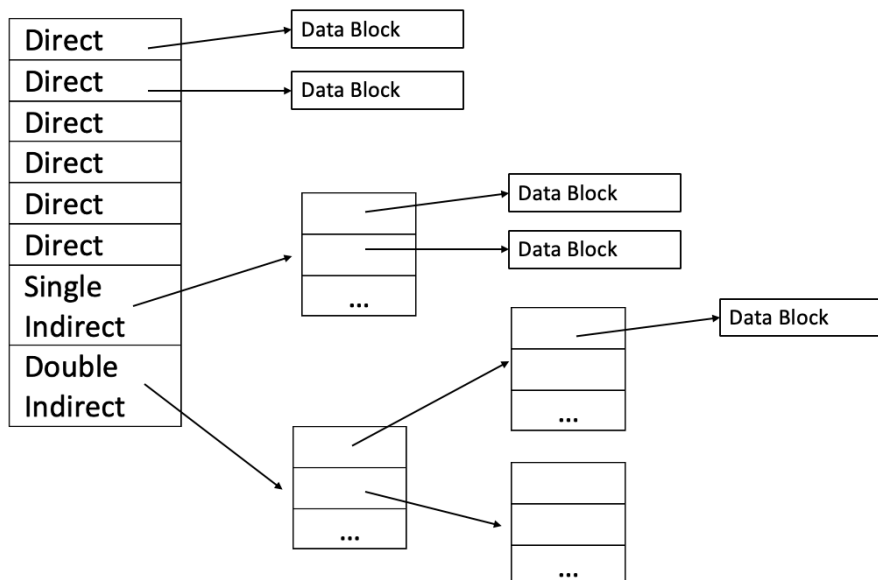
- d. Assuming that all block numbers in our filesystem is represented by a 24-bit number. Further, we assume that data blocks that are used as index blocks will only contain block numbers and nothing else. What is the maximum number of index/data blocks that a single index block can point to?

Block size = 2048 bytes.

Pointer size = 3 bytes

Maximum number of pointers = 682 pointers.

Our inode structure is illustrated below (not all data/index blocks are shown). There are six direct pointers, one single indirect pointer, and one double indirect pointer.



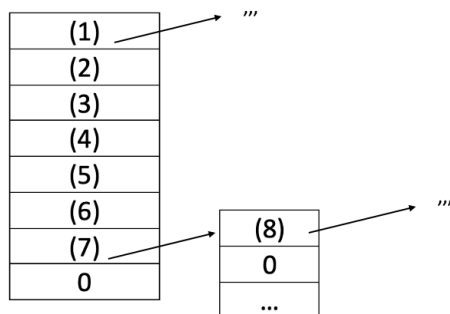
- e. What is the maximum file size in MB possible with this structure? State your answer correct to 2 decimal places. 1 MB = 1024 KB, 1 KB = 1024 bytes.

of direct blocks = 6

of single indirect blocks = 682

of double indirect blocks = 682^2
 Total = $6 + 682 + 682^2 = 465812$ blocks
 Size in KB = $465812 \times 2 = 931624$
 Size in MB = 909.79 MB

- f. The diagram below shows the inode structure. The (1), (2) etc. in the structures are labels that correspond to blanks that you need to fill in. We run the program above to completion. Fill in the block numbers of the data blocks used by the program in the blanks provided. If an entry in the inode is not pointing to any data block, fill in 0.



(1) 0 to 2047
 (2) 2048 to 4095
 (3) 4096 to 6143
 (4) 6144 to 8191
 (5) 8192 to 10240
 (6) 10240 to 12287
 (7) First index block, end at 14355

```
buffer=[0]
buffer1="Hello"
buffer2= "world."
```

```
fd = open("test.txt");
```

```
for i in range(1024):
    write(fd, buffer1, 5); // This will occupy 5120 bytes =
                          // 3 direct blocks
```

```
lseek(fd, 8192, CURR)      // This will go to 13312, which is
                          //the first indirect data block
```

```
for i in range(128):
    write(fd, buffer2, 6);
```

Map:

```
0b00000000 0b00000001 0b11111100 0b00001100
0b00011111 0b11111111 0b00001100 0b00000000
0b11001111 0b00000000 0b11110000 0b00001111
0b11111111 0b10101010 0b00001111 0b10101111
```

3 data blocks are 47, 48, 49

Index block is 50

Data block is 51

(1): 47

(2): 48

(3): 49

(4): 0

(5): 0

(6): 0

(7): 50

(8): 51